FASCICULE DE TRAVAUX PRATIQUES

IoT Appli et Plates formes
Fascicule de TP

12 laboratoires pratiques — du simulateur Python à AWS Cloud et au TinyML

LCS3-GLSI — ISIMa

Dr. Ahmed Bannour, MA en Télécommunications

Document compagnon du support de cours PowerPoint

2026-2027

Sommaire

Lab	Titre	Durée
TP1	Prise en main de MQTT en ligne de commande	1h30
TP2	Construire un simulateur de dispositif AIoT en Python	1h30
TP3	Rendre le dispositif réaliste : état, bruit, dérive et scénarios	1h00
TP4	Connecter le dispositif Python au broker MQTT	1h00
TP5	Communication bidirectionnelle : commandes et accusés de réception	1h00
TP6	Sécuriser la communication MQTT (TLS, authentification, ACL)	1h30
TP7	Persister la télémétrie avec MongoDB et l'analyser (agrégats, alertes)	2h00
TP8	Exposer le dispositif via une API REST (FastAPI)	1h30
TP9	AWS IoT Core + DynamoDB : du device au cloud	2h00
TP10	Traitement en flux avec AWS Kinesis et détection d'anomalies	2h00
TP11	TinyML : reconnaissance de gestes sur données de capteur	1h30
TP12	Projet de synthèse : concevoir une maison intelligente (Smart Home)	3h00

Comment utiliser ce fascicule

Chaque travail pratique (TP) est structuré de la même manière : objectifs pédagogiques, prérequis, schéma explicatif, étapes numérotées avec code et commandes, puis résultat attendu et questions de vérification.

Les TP suivent la progression du cours : ils s'appuient sur le travail réalisé dans le TP précédent. Il est recommandé de les réaliser dans l'ordre.

Environnement recommandé

- **Python 3.8** ou supérieur, avec un environnement virtuel dédié (venv)
- Éditeur de code : **VS Code** (ou équivalent)
- Broker **MQTT Mosquitto** installé localement
- Un compte AWS **Free Tier** pour les **TP 9 et 10**
- **MongoDB** Community Server (ou Docker) pour les **TP 7 et 8**
- **FastAPI + uvicorn** (pip install fastapi uvicorn) pour le TP8

TP 1 — Prise en main de MQTT en ligne de commande

⏱ Durée estimée : 1h30 | Niveau : Débutant

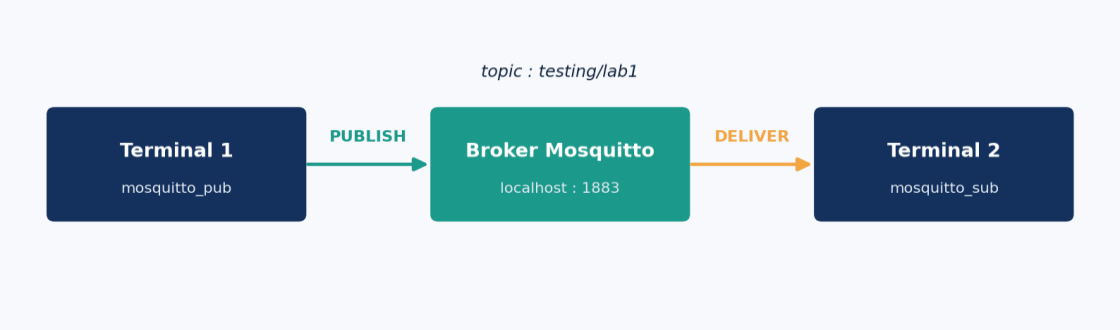
Objectifs pédagogiques

- Installer et démarrer un broker MQTT (Mosquitto) en local
- Publier et souscrire à des messages via la ligne de commande
- Comprendre la hiérarchie des topics et les wildcards **(+ et #)**
- Configurer le listener du broker (port, interface réseau)

Prérequis

- Un terminal / invite de commande
- Droits d'installation sur la machine

Schéma de l'architecture du TP



Deux terminaux se connectent au même broker Mosquitto : le premier publie des messages, le second s'y abonne et les reçoit en temps réel.

Étape 1 — Installer et démarrer le broker Mosquitto

1 Télécharger et installer Mosquitto

Rendez-vous sur https://www.mosquitto.org/download/ et installez la version correspondant à votre système d'exploitation.

2 Vérifier que le broker écoute bien sur le port 1883

Par défaut, Mosquitto écoute sur l'adresse 127.0.0.1, port 1883.

```
# Windows
$ netstat -an | findstr 1883

# Linux
$ netstat -ltn
tcp   0   0  0.0.0.0:1883   0.0.0.0:*   LISTEN

# macOS
$ lsof -nP -iTCP:1883 | grep LISTEN
```

Étape 2 — Publier et s'abonner en ligne de commande

3 **Publier un message**

```
$ mosquitto_pub -t "testing/lab1" -m "Testing publish"
```

Aucun résultat visible pour l'instant : personne n'est encore abonné à ce topic.

4 **S'abonner à un topic (dans un second terminal)**

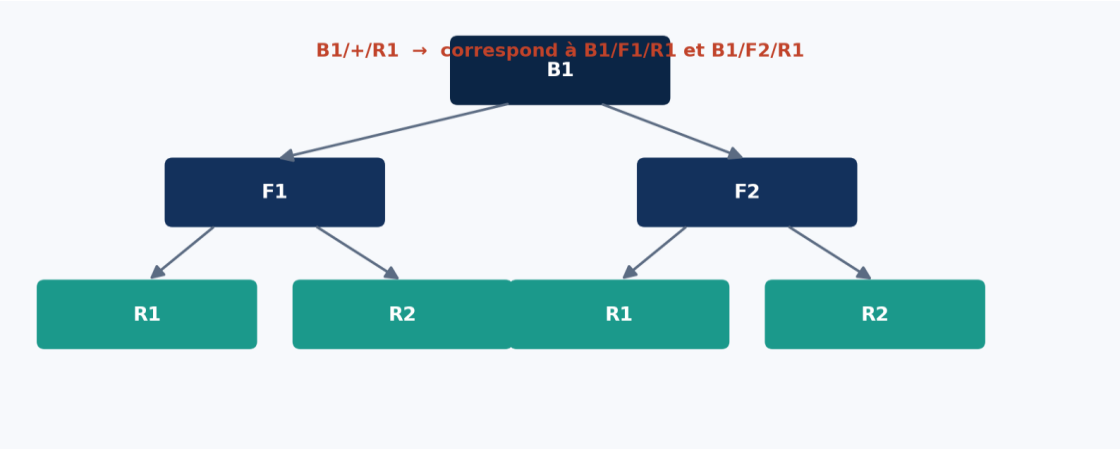
```
$ mosquitto_sub -t "testing/lab1"
```

Retournez dans le premier terminal et republiez le message : il doit maintenant s'afficher dans le second terminal. La commande mosquitto_sub reste bloquante et continue d'écouter les nouveaux messages.

Étape 3 — Approfondir les topics et les wildcards

Les topics sont des chaînes hiérarchiques séparées par « / » (ex. devices/building1/floor3/room8/temperature). Deux wildcards existent :

- + (niveau unique) — correspond à un seul niveau de la hiérarchie
- # (multi-niveaux) — correspond à tous les sous-niveaux, utilisé en fin de topic uniquement



5 **Tester le wildcard + (niveau unique)**

```
# Terminal 1
$ mosquitto_sub -t "B1/+/R1"

# Terminal 2
```

```
$ mosquitto_pub -t "B1/F1/R1" -m "Message to B1/F1/R1"
$ mosquitto_pub -t "B1/F2/R1" -m "Message to B1/F2/R1"
```

6 **Tester le wildcard # (multi-niveaux)**

```
# Terminal 1
$ mosquitto_sub -t "B1/#"

# Terminal 2
$ mosquitto_pub -t "B1/F1" -m "Message to B1/F1"
$ mosquitto_pub -t "B1/F2" -m "Message to B1/F2"
```

À retenir

- Les topics commençant par $ (ex. $SYS/broker/uptime) sont réservés aux statistiques internes du broker.
- Ne jamais s'abonner à # en production : cela génère un trafic incontrôlé sur un système à grande échelle.

Étape 4 — Publier un fichier et activer le mode debug

7 **Publier le contenu d'un fichier JSON**

Créez un fichier **message.json** avec le contenu suivant :

```
{
  "device_name":
"GL248",
  "device_type":
"counter",
  "timestamp": "2021-
01-14 13:13:13",
  "value": 45
}

$ mosquitto_pub -t
"filepublish" -f
message.json
```

8 **Activer le mode debug (-d) et assigner un Client ID (-i)**

```
$ mosquitto_sub -d -i
"terminal_subscriber"
-t "topic_name"
Client
terminal_subscriber
sending CONNECT
Client
terminal_subscriber
received CONNACK (0)
Subscribed (mid: 1):
0
```

Un Client ID explicite facilite grandement le débogage lorsqu'il y a plusieurs clients connectés.

Étape 5 — Configurer le listener du broker

9 **Modifier le fichier de configuration mosquitto.conf**

Emplacement du fichier selon l'OS : Windows → C:\Program Files\mosquitto\ · Linux → /etc/mosquitto/ · macOS → /usr/local/etc/mosquitto/

Sauvegardez d'abord le fichier original, puis remplacez son contenu par :

```
listener 5000
allow_anonymous true
```

Redémarrez le service (Gestionnaire de tâches sous Windows, systemctl sous Linux, kill+relance sous macOS), puis testez sur le nouveau port :

```
$ mosquitto_pub
-p 5000 -t
"topic_name" -m
"message"
```

```
$ mosquitto_sub -p 5000 -t "topic_name"
```

10 **Restreindre l'accès à une interface réseau**

```
# Broker accessible uniquement en local
listener 1883 localhost
allow_anonymous true

# Broker accessible sur le réseau local (LAN)
listener 1883 192.168.1.105
allow_anonymous true
```

Résultat attendu

- Le broker Mosquitto fonctionne en local et répond aux commandes publish/subscribe
- Vous savez utiliser les wildcards + et # pour filtrer les topics
- Vous savez reconfigurer le port et l'interface d'écoute du broker

Questions de vérification

- ☐ Quelle est la différence entre le wildcard + et le wildcard # ?
- ☐ Pourquoi mosquitto_sub est-elle une commande bloquante ?
- ☐ Quel est le risque de s'abonner directement au topic # dans un système en production ?
- ☐ À quoi sert le Client ID (-i) lors du débogage ?

TP 2 — Construire un simulateur de dispositif AIoT en Python

⏱ Durée estimée : 1h30 Niveau : Débutant / Intermédiaire

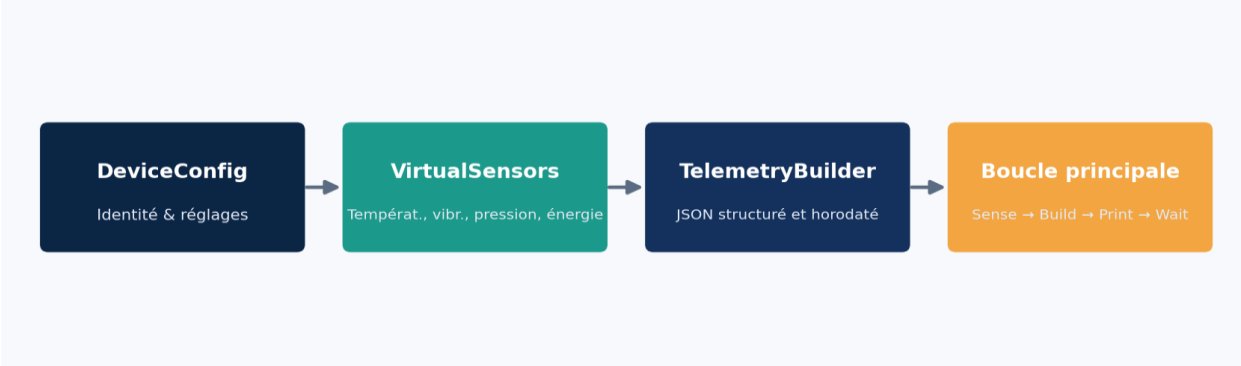
Objectifs pédagogiques

- Créer l'identité et la configuration d'un dispositif virtuel
- Simuler plusieurs capteurs avec des valeurs réalistes
- Construire une télémétrie structurée au format JSON
- Faire fonctionner le dispositif en continu via une boucle de simulation

Prérequis

- Python 3.8 ou supérieur installé
- Un environnement virtuel dédié (recommandé)
- Éditeur de code (VS Code ou équivalent)

Schéma de l'architecture du simulateur



Étape 1 — Préparer le projet

1 **Créer la structure du projet**

```
$ mkdir aiot-device-simulator && cd aiot-device-simulator
$ python -m venv .venv
# Windows
$ .venv\Scripts\activate
# Mac / Linux
$ source .venv/bin/activate
```

Créez les fichiers suivants : device_simulator.py (programme principal), config.py (configuration), sensors.py, telemetry.py. Seules des bibliothèques standard Python sont nécessaires à ce stade (time, random, json, datetime).

Étape 2 — Identité et configuration du dispositif

2 **Créer config.py**

Chaque dispositif IoT doit avoir une identité unique : ID, type, localisation, intervalle d'échantillonnage.

```
import time
from dataclasses import dataclass, field

@dataclass
class DeviceConfig:
    device_id: str = "MCH-007"
    device_type: str = "Bottle Filling Machine"
    location: str = "Production Line 2"
    sampling_interval: int = 5  # secondes
    status: str = "RUNNING"
    created_at: float = field(default_factory=time.time)

config = DeviceConfig()
```

Pourquoi une identité claire est essentielle

- Identification unique parmi des milliers de dispositifs
- Contexte (localisation, type) utile au routage et à l'analytique
- Comportement prévisible grâce à une configuration cohérente
- Horodatage de création utile à la traçabilité et au dépannage

Étape 3 — Simuler les capteurs virtuels

3 **Créer sensors.py**

La machine (MCH-007) possède quatre capteurs : température (70-85°C), vibration (1.5-3.5 mm/s), pression (5.0-7.0 bar), énergie (1.0-2.0 kWh).

```
import random
from typing import Dict

class VirtualSensors:
    def read_temperature(self) -> float:
        return round(random.uniform(70.0, 85.0), 2)

    def read_vibration(self) -> float:
        return round(random.uniform(1.5, 3.5), 2)

    def read_pressure(self) -> float:
        return round(random.uniform(5.0, 7.0), 2)

    def read_energy(self) -> float:
        return round(random.uniform(1.0, 2.0), 2)

    def read_all(self) -> Dict[str, float]:
        return {
            "temperature": self.read_temperature(),
            "vibration": self.read_vibration(),
            "pressure": self.read_pressure(),
            "energy": self.read_energy(),
        }
```

Étape 4 — Construire la télémétrie structurée

4 **Créer telemetry.py**

La télémétrie transforme les lectures brutes en message JSON structuré et horodaté, le format standard des systèmes IoT.

```
import time
from typing import Dict

class TelemetryBuilder:
    def __init__(self, device_config):
        self.cfg = device_config

    def build(self, readings: Dict[str, float]) -> Dict:
        return {
            "device_id": self.cfg.device_id,
            "device_type": self.cfg.device_type,
            "location": self.cfg.location,
            "timestamp": time.strftime(
                "%Y-%m-%dT%H:%M:%SZ", time.gmtime()),
```

```
            "status": self.cfg.status,
            "readings": {**readings},
        }
```

Exemple de message produit :

```
{
  "device_id": "MCH-007",
  "device_type": "Bottle Filling Machine",
  "location": "Production Line 2",
  "timestamp": "2025-05-11T10:30:15Z",
  "status": "RUNNING",
  "readings": {
    "temperature": 74.23,
    "vibration": 2.16,
    "pressure": 6.21,
    "energy": 1.37
  }
}
```

Étape 5 — La boucle de simulation

5 **Créer device_simulator.py**

Un dispositif IoT ne s'exécute pas une seule fois : il détecte, traite et communique en continu.

```
import time
from sensors import VirtualSensors
from telemetry import
TelemetryBuilder
from config import config

sensors = VirtualSensors()
telemetry_builder =
TelemetryBuilder(config)

print(f"\nDémarrage du dispositif
virtuel : {config.device_id}")
print(f"Échantillonnage toutes les
{config.sampling_interval}s\n")

try:
    while True:
        readings =
sensors.read_all()
```

```
        telemetry =
telemetry_builder.build(readings)
        print(telemetry)

time.sleep(config.sampling_interval)
except KeyboardInterrupt:
    print("\nDispositif arrêté.")
```

6 **Exécuter le simulateur**

```
$ python device_simulator.py
```

Le programme doit afficher une nouvelle ligne de télémétrie toutes les 5 secondes, avec des valeurs légèrement différentes à chaque cycle. Arrêtez avec Ctrl+C.

Résultat attendu

- Un dispositif virtuel fonctionnel avec une identité claire
- Quatre capteurs générant des données réalistes et variables
- Une télémétrie structurée au format JSON standard
- Une boucle continue produisant des données toutes les 5 secondes

Questions de vérification

- ☐ Pourquoi utiliser une dataclass plutôt que des variables globales pour la configuration ?
- ☐ Que se passerait-il si sampling_interval était fixé à 0 ?
- ☐ Quels champs ajouteriez-vous à la télémétrie pour détecter un état « WARNING » ?

TP 3 — Rendre le dispositif réaliste : état, bruit, dérive et scénarios

Durée estimée : 1h00 Niveau : Intermédiaire

Objectifs pédagogiques

- Simuler des capteurs à état (valeur qui évolue dans le temps, pas purement aléatoire)
- Introduire du bruit (noise) et une dérive (drift) réalistes sur chaque lecture
- Évaluer l'état de la machine (RUNNING / WARNING / FAULT) à partir de seuils métier
- Rejouer des scénarios contrôlés pour démontrer chaque état

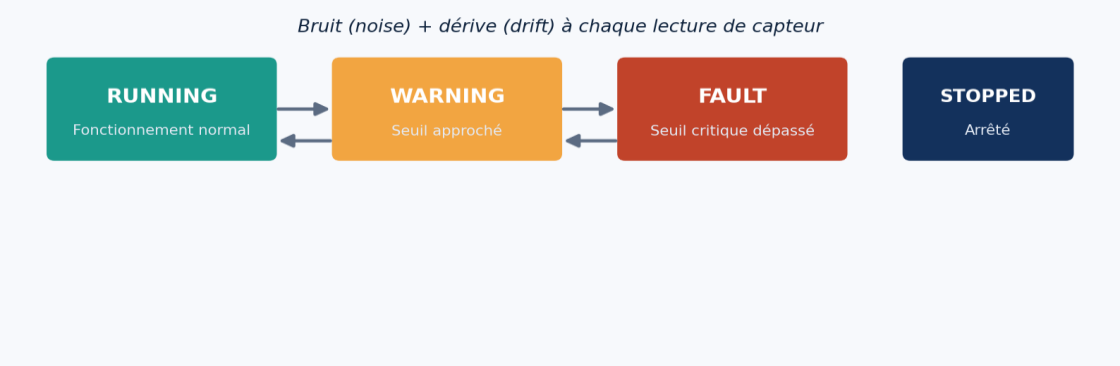
Prérequis

- TP2 terminé (simulateur Python de base)
- Le dossier du TP2 comme point de départ

Pourquoi cette étape est essentielle

- Un dispositif IoT réel ne produit pas des valeurs indépendantes à chaque lecture : la température d'une machine évolue progressivement, avec du bruit de mesure autour d'une tendance.
- Ce TP transforme le simulateur "aléatoire" du TP2 en un dispositif qui se comporte comme une vraie machine.

Schéma des états de la machine



Étape 1 — Modéliser un capteur à état (bruit + dérive)

1 Créer sensors.py

Contrairement au TP2 (valeur purement aléatoire à chaque appel), chaque capteur conserve maintenant sa dernière valeur, qui dérive légèrement (drift) puis reçoit un bruit de mesure (noise).

```
import random
from dataclasses import dataclass

@dataclass
class SensorModel:
    name: str
    value: float
    min_value: float
```

```
    max_value: float
    drift: float
    noise: float

    def read(self):
        # Dérive progressive de la valeur réelle
        self.value += random.uniform(-self.drift, self.drift)
        self.value = max(self.min_value, min(self.max_value, self.value))
        # Bruit de mesure ajouté à la lecture
        observed = self.value + random.uniform(-self.noise, self.noise)
        return round(observed, 2)

class VirtualSensors:
    def __init__(self):
        self.temperature = SensorModel("temperature", 72.0, 65.0, 95.0,
0.35, 0.20)
        self.vibration = SensorModel("vibration", 2.0, 1.0, 5.0, 0.08,
0.05)
        self.pressure = SensorModel("pressure", 6.2, 4.0, 7.5, 0.08,
0.04)
        self.energy = SensorModel("energy", 1.4, 0.8, 2.5, 0.04, 0.03)

    def read_all(self):
        return {
            "temperature": self.temperature.read(),
            "vibration": self.vibration.read(),
            "pressure": self.pressure.read(),
            "energy": self.energy.read(),
        }
```

Drift vs Noise

- Drift : la valeur réelle du système évolue lentement dans le temps (usure, échauffement progressif).
- Noise : à chaque mesure, un petit écart aléatoire s'ajoute autour de la valeur réelle (imprécision du capteur).

Étape 2 — Définir les seuils métier et évaluer l'état

2 **Créer config.py avec les seuils**

```
DEVICE_CONFIG = {
    "device_id": "MCH-007",
    "device_type": "Bottle Filling
Machine",
    "location": "Production Line 2",
    "sampling_interval": 2,
```

```
    "status": "RUNNING",
}

THRESHOLDS = {
    "temperature_warning": 80.0,
    "temperature_fault": 88.0,
    "vibration_warning": 3.2,
    "vibration_fault": 4.0,
    "pressure_warning_low": 4.5,
    "pressure_fault_low": 4.0,
}
```

3 **Créer state.py**

```
from enum import Enum
from config import THRESHOLDS

class MachineState(str, Enum):
    RUNNING = "RUNNING"
    WARNING = "WARNING"
    FAULT = "FAULT"
    STOPPED = "STOPPED"

def evaluate_state(readings):
    t = readings["temperature"]
    v = readings["vibration"]
    p = readings["pressure"]

    if t >=
THRESHOLDS["temperature_fault"] and
v >= THRESHOLDS["vibration_fault"]:
        return MachineState.FAULT,
"Critical temperature and vibration"
    if t >=
THRESHOLDS["temperature_fault"]:
        return MachineState.FAULT,
"Critical temperature"
    if p <=
THRESHOLDS["pressure_fault_low"]:
        return MachineState.FAULT,
"Critical low pressure"
    if (t >=
THRESHOLDS["temperature_warning"]
            or v >=
THRESHOLDS["vibration_warning"]
```

```
            or p <=
THRESHOLDS["pressure_warning_low"]):
        return MachineState.WARNING,
"Operating parameter approaching
limit"

    return MachineState.RUNNING,
"Normal operation"
```

Remarquez la hiérarchie des règles : les conditions FAULT sont testées avant WARNING, afin qu'un cas critique ne soit jamais masqué par un cas moins grave.

Étape 3 — Rejouer des scénarios contrôlés

4 **Créer scenarios.py**

Pour démontrer chaque état de manière fiable (plutôt que d'attendre le hasard), on définit des jeux de valeurs fixes représentant des situations réelles.

```
SCENARIOS = {
    "NORMAL": {"temperature": 72.0, "vibration": 2.0, "pressure": 6.2,
"energy": 1.4},
    "OVERHEATING": {"temperature": 86.0, "vibration": 2.8, "pressure":
6.1, "energy": 1.7},
    "COMBINED_FAULT": {"temperature": 91.0, "vibration": 4.6, "pressure":
6.5, "energy": 2.1},
    "LOW_PRESSURE": {"temperature": 74.0, "vibration": 2.2, "pressure":
4.2, "energy": 1.5},
}
```

Essayez de faire passer evaluate_state() directement les valeurs de chaque scénario : vérifiez que OVERHEATING déclenche bien un WARNING, et que COMBINED_FAULT déclenche bien un FAULT.

Étape 4 — Construire une télémétrie consciente du contexte

5 **Créer telemetry.py**

```
from datetime import datetime

def build_telemetry(device_config, readings,
state, message):
    return {
        "device_id":
device_config["device_id"],
        "device_type":
device_config["device_type"],
        "location":
device_config["location"],
```

```
        "timestamp":
datetime.now().isoformat(timespec="seconds"),
        "state": state.value,
        "message": message,
        "readings": readings,
    }
```

6 — **Assembler le tout dans device_simulator.py**

```
from config import DEVICE_CONFIG
from sensors import VirtualSensors
from state import evaluate_state
from telemetry import build_telemetry
import time

class AIoTDevice:
    def __init__(self):
        self.config = DEVICE_CONFIG.copy()
        self.sensors = VirtualSensors()

    def read_and_process(self):
        readings = self.sensors.read_all()
        state, message =
evaluate_state(readings)
        return build_telemetry(self.config,
readings, state, message)

    def run(self, cycles=10):
        print(f"Démarrage de
{self.config['device_id']}...")
        for cycle in range(1, cycles + 1):
            telemetry =
self.read_and_process()
            print(f"\nCycle {cycle} — État :
{telemetry['state']} —
{telemetry['message']}")
            print(telemetry["readings"])

time.sleep(self.config["sampling_interval"])

if __name__ == "__main__":
    AIoTDevice().run(cycles=10)
```

Exécutez le programme et laissez-le tourner plusieurs minutes : observez la valeur de température dériver progressivement plutôt que sauter aléatoirement d'un cycle à l'autre.

Résultat attendu

- Des capteurs qui dérivent dans le temps plutôt que de sauter aléatoirement
- Une machine à états claire : RUNNING → WARNING → FAULT selon des seuils métier
- Des scénarios rejouables pour démontrer chaque état de façon fiable
- Une télémétrie enrichie d'un état et d'un message explicite

Questions de vérification

- ☐ Pourquoi teste-t-on les conditions FAULT avant les conditions WARNING dans evaluate_state ?
- ☐ Quelle est la différence de rôle entre le paramètre drift et le paramètre noise ?
- ☐ Comment ajouteriez-vous un état MAINTENANCE déclenché manuellement plutôt que par un seuil ?

TP 4 — Connecter le dispositif Python au broker MQTT

⏱ Durée estimée : 1h00 Niveau : Intermédiaire

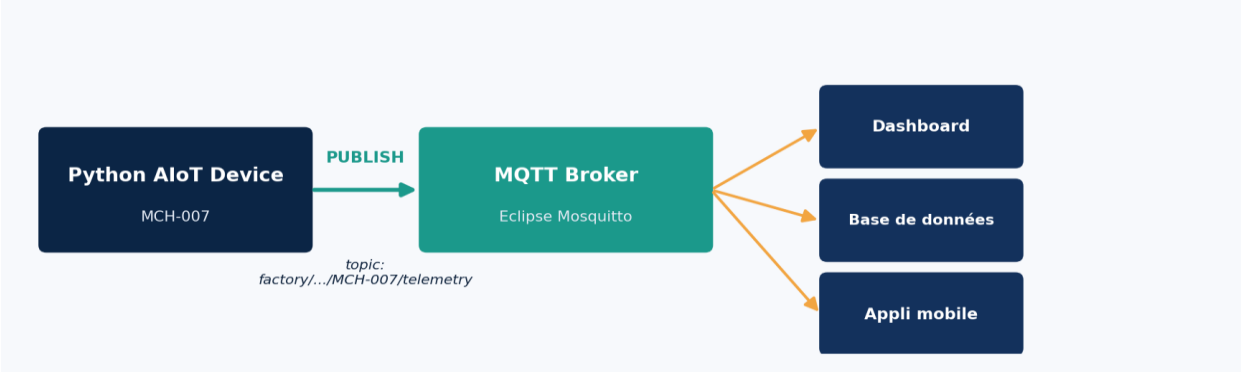
Objectifs pédagogiques

- Installer et utiliser la bibliothèque Paho MQTT
- Connecter le simulateur Python au broker Mosquitto
- Concevoir un topic structuré et cohérent pour le dispositif
- Publier la télémétrie en temps réel avec la Qualité de Service adaptée

Prérequis

- TP1 (broker Mosquitto opérationnel) et TP2 (simulateur Python) terminés
- Le broker Mosquitto doit être démarré (localhost:1883)

Schéma de l'architecture du TP



Le dispositif ne connaît pas ses abonnés : il publie sur un topic, le broker se charge de la distribution vers dashboards, bases de données ou applications mobiles.

Étape 1 — Installer le client MQTT Python

1 **Installer Paho MQTT**

```
$ pip install paho-mqtt
```

Étape 2 — Concevoir le topic du dispositif

Un topic bien structuré rend les données organisées, scalables et faciles à gérer :

Objectif	Topic	Utilisation
Télémétrie	factory/production-line-2/machine/MCH-007/telemetry	Mesures et données capteurs
État / santé	factory/production-line-2/machine/MCH-007/status	État et santé du dispositif
Commandes	factory/production-line-2/machine/MCH-007/command	Envoi de commandes au dispositif
Alertes	factory/production-line-2/machine/MCH-007/event	Alertes, alarmes, événements

Étape 3 — Se connecter au broker

2 **Créer device_mqtt_connect.py**

```
import paho.mqtt.client as mqtt

BROKER_HOST = "localhost"
BROKER_PORT = 1883
KEEPALIVE = 60

def on_connect(client, userdata, flags, rc):
    if rc == 0:
        print("Connecté au broker MQTT")
    else:
        print(f"Échec de connexion, code : {rc}")

client = mqtt.Client(client_id="MCH-007-Device")
client.on_connect = on_connect

print("Connexion au broker...")
client.connect(BROKER_HOST, BROKER_PORT, KEEPALIVE)
```

Astuce

– Pour un broker distant, remplacez "localhost" par l'adresse IP ou l'URL du

broker, et vérifiez que le port est ouvert.

Étape 4 — Publier la télémétrie

3 — **Ajouter la fonction de publication**

```
import json

TOPIC_TELEMETRY =
"factory/production-line-
2/machine/MCH-007/telemetry"

def publish_telemetry(client,
telemetry):
    payload = json.dumps(telemetry)
    result =
client.publish(TOPIC_TELEMETRY,
payload, qos=1)
    if result.rc == 0:
        print(f"Télémétrie publiée
sur {TOPIC_TELEMETRY}")
    else:
        print(f"Échec de
publication, code : {result.rc}")
```

4 — **Intégrer la publication dans la boucle du simulateur (TP2)**

```
client.loop_start()
client.connect(BROKER_HOST,
BROKER_PORT, KEEPALIVE)

while True:
    readings = sensors.read_all()
    telemetry =
telemetry_builder.build(readings)
    publish_telemetry(client,
telemetry)

time.sleep(config.sampling_interval)
```

Le paramètre qos=1 garantit une livraison « au moins une fois » — un choix adapté à de la télémétrie régulière (voir le cours, Module 3, sur les niveaux de QoS).

Étape 5 — Vérifier la réception des messages

5 **S'abonner au topic depuis un terminal**

```
$ mosquitto_sub -h localhost -t "factory/production-line-2/machine/MCH-007/telemetry" -v
```

Lancez ensuite votre script Python dans un autre terminal : les messages JSON doivent apparaître en temps réel dans le terminal abonné.

Résultat attendu

- Le dispositif Python publie sa télémétrie toutes les 5 secondes sur MQTT
- Le topic suit une structure hiérarchique claire et réutilisable
- La chaîne complète Device → Broker → Subscriber fonctionne de bout en bout

Questions de vérification

- ☐ Pourquoi choisir QoS 1 plutôt que QoS 0 ou QoS 2 pour cette télémétrie ?
- ☐ Que se passe-t-il si deux dispositifs utilisent le même client_id ?
- ☐ Comment ajouteriez-vous un topic de commande pour piloter le dispositif à distance ?

TP 5 — Communication bidirectionnelle : commandes et accusés de réception

Durée estimée : 1h00 Niveau : Intermédiaire

Objectifs pédagogiques

- Faire écouter le dispositif sur un topic de commande, en plus de publier sa télémétrie
- Exécuter des commandes à distance (START, STOP, STATUS)
- Répondre à chaque commande par un accusé de réception (ACK) traçable
- Rejeter proprement une commande invalide ou non reconnue

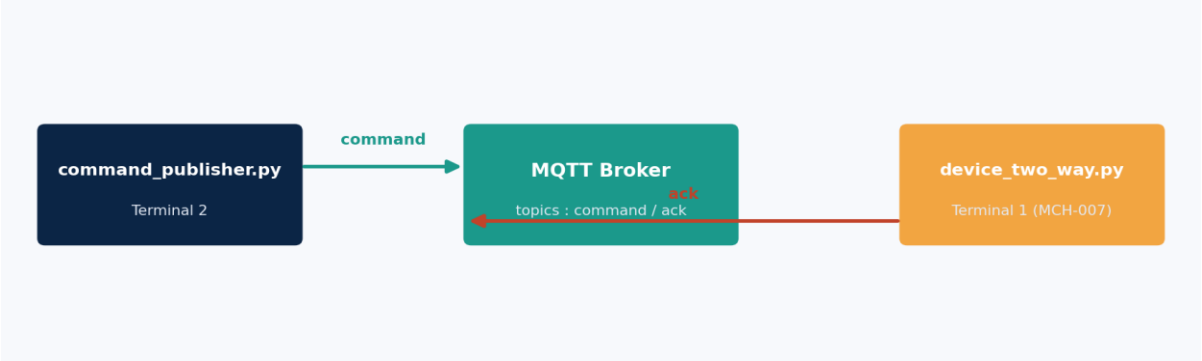
Prérequis

- TP4 terminé (dispositif connecté et publiant sa télémétrie)
- Broker MQTT Mosquitto opérationnel en local

Pourquoi aller au-delà de la simple publication

- Jusqu'ici, notre dispositif ne fait que parler (publier sa télémétrie). Un vrai système IoT d'entreprise doit aussi pouvoir écouter : recevoir des ordres d'arrêt, de redémarrage, ou de changement de configuration.
- C'est ce qui transforme un capteur passif en un dispositif véritablement pilotable à distance.

Schéma de la communication bidirectionnelle



Le dispositif publie toujours sa télémétrie en continu, mais s'abonne désormais aussi à un topic de commande. Chaque commande reçue déclenche une réponse (ACK) publiée sur un troisième topic dédié.

Étape 1 — Concevoir les 3 topics du dispositif

Topic	Sens	Rôle
factory/.../MCH-007/telemetry	Device → Broker	Télémétrie publiée en continu (TP4)
factory/.../MCH-007/command	Application → Device	Commandes envoyées au dispositif
factory/.../MCH-007/ack	Device → Application	Accusé de réception de chaque commande

Étape 2 — Construire le dispositif à double sens

1 — **Créer config.py**

```
DEVICE_CONFIG = {
    "device_id": "MCH-007",
    "device_type": "Bottle Filling Machine",
    "location": "Production Line 2",
    "sampling_interval": 2,
}

MQTT_CONFIG = {
    "broker_host": "localhost",
    "broker_port": 1883,
    "keepalive": 60,
    "telemetry_topic": "factory/production-line-
2/machine/MCH-007/telemetry",
    "command_topic": "factory/production-line-
2/machine/MCH-007/command",
    "ack_topic": "factory/production-line-
2/machine/MCH-007/ack",
    "qos": 0,
    "retain": False,
}
```

2 — **Créer device_two_way.py — se connecter et s'abonner aux commandes**

```
import json, time, uuid
import paho.mqtt.client as mqtt
from config import DEVICE_CONFIG, MQTT_CONFIG
from state import MachineState,
evaluate_sensor_state
from telemetry import build_telemetry, build_ack

SUPPORTED_COMMANDS = {"START", "STOP", "STATUS"}

class AIoTDeviceTwoWay:
    def __init__(self):
        self.state = MachineState.RUNNING
        self.state_message = "Normal operation"
        self.client =
mqtt.Client(client_id=DEVICE_CONFIG["device_id"])
        self.client.on_connect = self.on_connect
        self.client.on_message = self.on_message
```

```
    def on_connect(self, client, userdata, flags,
rc):

client.subscribe(MQTT_CONFIG["command_topic"])
        print("Connecté et abonné :",
MQTT_CONFIG["command_topic"])
```

La méthode **on_connect()** s'abonne au topic de commande dès que la connexion est établie — c'est ce qui permet au dispositif de recevoir des ordres, en plus de publier sa télémétrie.

Étape 3 — Traiter une commande reçue

3	Implémenter on_message()

```
   def on_message(self, client,
userdata, message):
        print("\nCommande reçue :",
message.payload.decode())
        try:
            data =
json.loads(message.payload.decode())
            command =
str(data.get("command", "")).upper()
            request_id =
data.get("request_id",
str(uuid.uuid4()))
        except
(json.JSONDecodeError,
UnicodeDecodeError):

self.publish_ack("UNKNOWN", "N/A",
"REJECTED", "Payload JSON invalide")
            return

        if command not in
SUPPORTED_COMMANDS:
            self.publish_ack(command
or "UNKNOWN", request_id,
"REJECTED", "Commande non
supportée")
            print("Commande rejetée
:", command)
            return

        if command == "START":
```

```
            self.state =
MachineState.RUNNING
            self.state_message =
"Démarré par commande distante"
        elif command == "STOP":
            self.state =
MachineState.STOPPED
            self.state_message =
"Arrêté par commande distante"
        # STATUS ne modifie pas
l'état : il déclenche simplement un
ACK

        self.publish_ack(command,
request_id, "SUCCESS")
        print(f"Commande exécutée :
{command} | État =
{self.state.value}")
```

Robustesse : valider avant d'agir

- On valide d'abord le format JSON, puis on vérifie que la commande fait partie des commandes supportées, avant de modifier le moindre état.
- Toute commande invalide ou inconnue est REJECTED avec une raison explicite — jamais ignorée silencieusement.

Étape 4 — Publier l'accusé de réception (ACK)

4	Construire et publier l'ACK

```
    def publish_ack(self, command,
request_id, status, reason=None):
        payload = {
            "device_id":
DEVICE_CONFIG["device_id"],
            "command": command,
            "request_id": request_id,
            "status": status,  # SUCCESS ou
REJECTED
        }
        if reason:
            payload["reason"] = reason
```

```
self.client.publish(MQTT_CONFIG["ack_topic"],
json.dumps(payload))
```

Chaque commande — acceptée ou rejetée — génère un accusé de réception traçable, avec le même request_id que la commande d'origine. C'est ce qui permet à l'application appelante de savoir si sa commande a bien été exécutée.

Étape 5 — Tester la communication bidirectionnelle

5 **Lancer le dispositif (Terminal 1)**

```
$ python
device_two_way.py
```

6 **Envoyer des commandes interactivement (Terminal 2)**

Créez command_publisher.py, qui demande une commande au clavier et la publie avec un request_id unique :

```
command = input("Commande
[START/STOP/STATUS/QUIT]: ").strip().upper()
request_id = f"REQ-
{uuid.uuid4().hex[:8].upper()}"
payload = {"command": command, "request_id":
request_id}
client.publish(MQTT_CONFIG["command_topic"],
json.dumps(payload))

$ python command_publisher.py
```

7 **Observer les accusés de réception (Terminal 3, optionnel)**

```
$ mosquitto_sub -t "factory/production-line-
2/machine/MCH-007/ack" -v
```

Envoyez STOP puis START depuis le Terminal 2, et observez dans le Terminal 1 le changement d'état du dispositif, et dans le Terminal 3 les accusés de réception correspondants.

Défi (optionnel)

Ajoutez une commande RESET qui ne soit acceptée que lorsque le dispositif est en état FAULT (sinon, elle doit être REJECTED avec une raison explicite).

Résultat attendu

- Le dispositif écoute désormais un topic de commande en plus de publier sa télémétrie
- Les commandes START/STOP/STATUS modifient l'état du dispositif à distance
- Chaque commande génère un accusé de réception traçable (SUCCESS ou REJECTED)
- Les commandes invalides sont rejetées proprement, sans jamais faire planter le dispositif

Questions de vérification

- ☐ Pourquoi valider le format et le contenu de la commande avant de modifier l'état du dispositif ?
- ☐ À quoi sert le request_id transmis entre la commande et son accusé de réception ?
- ☐ Que se passerait-il si deux applications envoyaient des commandes contradictoires en même temps ?

TP 6 — Sécuriser la communication MQTT (TLS, authentification, ACL)

⏱ Durée estimée : 1h30 Niveau : Intermédiaire

Objectifs pédagogiques

- Comprendre les 3 piliers de la sécurité MQTT : authentification, chiffrement, autorisation
- Activer le **chiffrement TLS** sur le broker Mosquitto
- Mettre en place une authentification par identifiants (username/password)
- Restreindre les droits de publication/souscription via une ACL

Prérequis

- TP1 terminé (broker Mosquitto opérationnel)
- OpenSSL installé (généralement préinstallé sous Linux/macOS ; disponible via Git Bash ou WSL sous Windows)

Schéma des 3 piliers de sécurité



Ces trois couches se combinent : l'authentification vérifie l'identité, le chiffrement protège la confidentialité du message, et l'autorisation limite ce que chaque client authentifié a le droit de faire.

Étape 1 — Générer les certificats pour le chiffrement TLS

1	Créer une autorité de certification (CA) locale

```
$ mkdir mqtt-tls && cd mqtt-tls
$ openssl genrsa -out ca.key 2048
$ openssl req -x509 -new -nodes -key ca.key -days 365 -out ca.crt -subj "/CN=Mon-AIoT-CA"
```

2 **Générer la clé et le certificat du serveur (broker)**

```
$ openssl genrsa -out server.key 2048
$ openssl req -new -key server.key -out server.csr -subj "/CN=localhost"
$ openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365
```

Ce que vous venez de créer

- ca.crt : le certificat racine de confiance, à distribuer aux clients
- server.key / server.crt : l'identité chiffrée du broker Mosquitto
- Dans un contexte de production, ces certificats seraient émis par une PKI d'entreprise

Étape 2 — Activer TLS et l'authentification sur Mosquitto

3 **Éditer mosquitto.conf**

```
# --- Chiffrement TLS ---
listener 8883
cafile /chemin/vers/mqtt-tls/ca.crt
```

```
certfile
/chemin/vers/mqtt-
tls/server.crt
keyfile
/chemin/vers/mqtt-
tls/server.key

# ---
Authentification -
--
allow_anonymous
false
password_file
/chemin/vers/mqtt-
tls/passwordfile
```

4 **Créer un identifiant utilisateur**

```
$ mosquitto_passwd
-c passwordfile
device_mch007
Password: ********
```

5 **Redémarrer le broker**

Redémarrez le service Mosquitto pour appliquer la nouvelle configuration (voir TP1, étape 5 pour la procédure selon votre OS).

Étape 3 — Tester la connexion sécurisée

6 **Publier via TLS avec identifiants**

```
$ mosquitto_pub
--cafile ca.crt
-h localhost -p
8883 -u
device_mch007 -P
<mot_de_passe> -
t
"testing/secure"
-m "Message
chiffré"
```

Sans le bon certificat ou les bons identifiants, la connexion doit être refusée — vérifiez-le en testant volontairement avec un mauvais mot de passe.

Étape 4 — Restreindre les droits avec une ACL

7 **Créer un fichier aclfile**

```
# aclfile : le device MCH-007 ne peut publier
# que sur son propre topic de télémétrie et lire ses commandes

user device_mch007
topic write factory/production-line-2/machine/MCH-007/telemetry
topic read factory/production-line-2/machine/MCH-007/command
```

8 **Référencer l'ACL dans mosquitto.conf**

```
acl_file /chemin/vers/mqtt-tls/aclfile
```

Redémarrez le broker, puis vérifiez que ce client ne peut plus publier sur un autre topic que le sien (ex. tentez de publier sur un topic d'une autre machine — cela doit échouer).

Étape 5 — Adapter le simulateur Python (TP4) à la connexion sécurisée

```
import paho.mqtt.client as mqtt

client = mqtt.Client(client_id="MCH-007-Device")
client.username_pw_set("device_mch007", "<mot_de_passe>")
client.tls_set(ca_certs="ca.crt")
client.connect("localhost", 8883, 60)
```

Résultat attendu

- Le broker n'accepte plus les connexions anonymes
- Le trafic MQTT est chiffré via TLS (port 8883)
- Chaque dispositif ne peut publier/souscrire qu'aux topics qui lui sont autorisés

Questions de vérification

- ☐ Quelle est la différence entre authentification et autorisation ?
- ☐ Pourquoi le chiffrement seul (TLS) ne suffit-il pas à sécuriser un système MQTT ?
- ☐ Que se passe-t-il si le certificat ca.crt n'est pas fourni côté client ?

TP 7 — Persister la télémétrie avec MongoDB et l'analyser

⏱ Durée estimée : 2h00 Niveau : Intermédiaire

Objectifs pédagogiques

- Installer MongoDB (localement ou via Docker) pour stocker la télémétrie
- Se connecter à MongoDB en Python avec PyMongo
- Concevoir un document MongoDB pour la télémétrie IoT
- Construire un service d'ingestion MQTT → MongoDB
- Calculer des agrégats quotidiens (min/max/moyenne) et déclencher des alertes par seuil

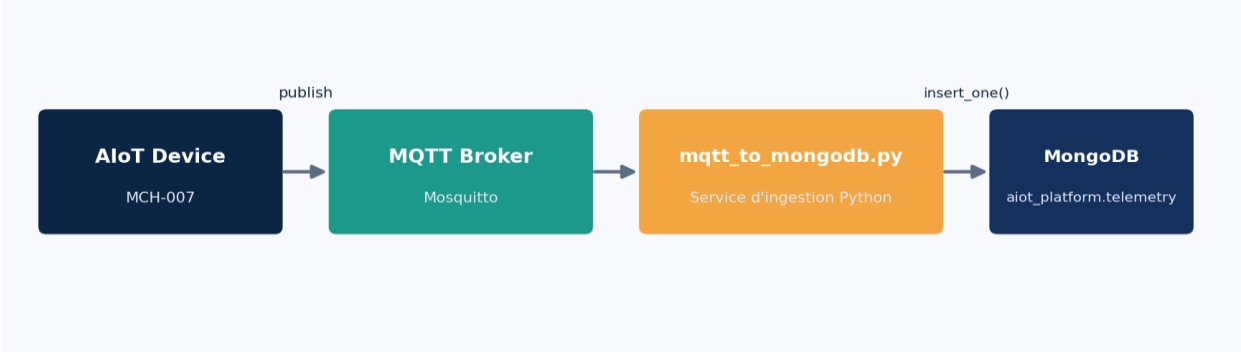
Prérequis

- TP4 terminé (dispositif publiant sur MQTT)
- MongoDB Community Server ou Docker installé
- Bibliothèque PyMongo (pip install pymongo)

Pourquoi une mémoire pour le dispositif ?

Aujourd'hui, le dispositif publie sa télémétrie via MQTT, mais rien ne la conserve : dès que l'application s'arrête, les données sont perdues. Impossible d'analyser une tendance ou un historique. MongoDB apporte cette mémoire persistante.

Schéma du flux d'ingestion



Étape 1 — Installer MongoDB

1 **Option A — Installation locale (Windows, PowerShell administrateur)**

```
$ winget install MongoDB.MongoDB
```

Le service démarre automatiquement sur le port 27017 et stocke ses données localement.

2 **Option B — Exécuter MongoDB via Docker (recommandé, portable)**

```
$ docker run -d --name
```

```
aiot-mongodb
-p
27017:27017 -
v mongo-
data:/data/db
mongo:latest
```

Bonnes pratiques

- Utiliser un volume nommé (-v) pour que les données persistent même si le conteneur redémarre
- En production, activer l'authentification et restreindre l'accès réseau

Étape 2 — Concevoir le document de télémétrie

Base : aiot_platform · Collection : telemetry · Chaque document représente un événement de télémétrie.

```
{
  "_id": ObjectId("..."),
  "device_id": "MCH-007",
  "timestamp": ISODate("2025-05-15T10:15:21.123Z"),
  "state": "RUNNING",
  "readings": {
    "temperature": 73.4,
    "vibration": 2.1,
    "pressure": 6.2,
    "energy": 1.82
  },
  "status": { "warning": false, "fault": false },
  "ingested_at": ISODate("2025-05-15T10:15:21.456Z"),
  "source": "mqtt_ingestion_service",
  "schema_version": "1.0"
}
```

Pourquoi cette structure ? Flexibilité de schéma (ajout de capteurs sans migration), champ timestamp adapté aux séries temporelles, structure imbriquée lisible, requêtes puissantes sur n'importe quel champ.

Étape 3 — Se connecter à MongoDB depuis Python

3 **Tester la connexion**

```
from pymongo import MongoClient

client =
MongoClient("mongodb://localhost:27017/")
db = client["aiot_platform"]
collection = db["telemetry"]

# Créer un index sur le timestamp pour
accélérer les requêtes
collection.create_index("timestamp")

print(client.list_database_names())
print("Connecté à MongoDB ☑")
```

Sortie attendue :

```
['admin', 'config', 'local']
Connecté à MongoDB ☑
```

Étape 4 — Construire le service d'ingestion MQTT → MongoDB

4 **Créer mqtt_to_mongodb.py**

```
import json
import paho.mqtt.client as mqtt
from pymongo import MongoClient

# Connexion MongoDB
client =
MongoClient("mongodb://localhost:27017/")
db = client["aiot_platform"]
collection = db["telemetry"]

# Callback appelé à chaque message MQTT
reçu
def on_message(client, userdata,
message):
    data =
json.loads(message.payload.decode())
    result = collection.insert_one(data)
    print(f"Télémétrie stockée, _id :
{result.inserted_id}")
```

```
# Configuration MQTT
broker = "localhost"
topic = "factory/production-line-
2/machine/MCH-007/telemetry"

client_mqtt = mqtt.Client()
client_mqtt.on_message = on_message
client_mqtt.connect(broker, 1883, 60)
client_mqtt.subscribe(topic)

print("Service d'ingestion en écoute...")
client_mqtt.loop_forever()
```

5 **Exécuter le pipeline complet**

Ouvrez trois terminaux :

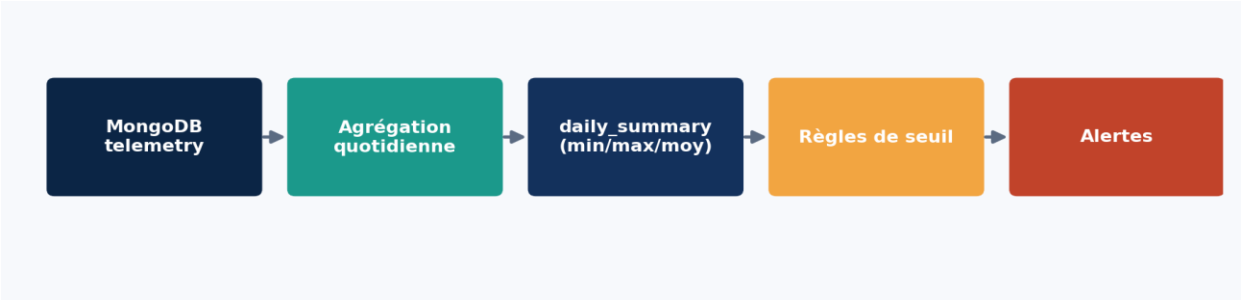
1. Terminal 1 : lancez votre simulateur Python du TP4 (python device_simulator.py)
2. Terminal 2 : lancez le service d'ingestion (python mqtt_to_mongodb.py)
3. Terminal 3 : ouvrez MongoDB Compass (ou mongosh) et observez les documents apparaître en temps réel dans aiot_platform.telemetry

Étape 5 — Vérifier avec MongoDB Compass

- ☐ Se connecter à mongodb://localhost:27017
- ☐ Ouvrir la base aiot_platform
- ☐ Ouvrir la collection telemetry et observer les documents
- ☐ Vérifier la présence de l'index sur timestamp

Étape 6 — Analyser la télémétrie : agrégats quotidiens

Stocker la télémétrie ne suffit pas : il faut aussi l'exploiter. On calcule ici un résumé quotidien (min / max / moyenne) par dispositif, stocké dans une nouvelle collection daily_summary.



6 **Créer daily_summary.py**

```
from pymongo import MongoClient
from datetime import datetime, timedelta
```

```
client =
MongoClient("mongodb://localhost:27017/")
db = client["aiot_platform"]
telemetry = db["telemetry"]
daily_summary = db["daily_summary"]

# Fenêtre : les dernières 24 heures
since = datetime.utcnow() -
timedelta(days=1)

pipeline = [
    {"$match": {"timestamp": {"$gte":
since}}},
    {"$group": {
        "_id": "$device_id",
        "min_temperature": {"$min":
"$readings.temperature"},
        "max_temperature": {"$max":
"$readings.temperature"},
        "avg_temperature": {"$avg":
"$readings.temperature"},
        "count": {"$sum": 1},
    }},
]

results =
list(telemetry.aggregate(pipeline))
for r in results:
    r["date"] =
datetime.utcnow().strftime("%Y-%m-%d")
    daily_summary.insert_one(r)
    print(f"{r['_id']} :
min={r['min_temperature']:.1f} "

f"max={r['max_temperature']:.1f} "

f"avg={r['avg_temperature']:.1f}
(n={r['count']})")
```

Pipeline d'agrégation MongoDB

- $match filtre les documents (ici, la fenêtre des dernières 24h)
- $group calcule min/max/moyenne par device_id, comme un GROUP BY en SQL
- Cette approche est bien plus efficace qu'une boucle Python sur tous les documents

Étape 7 — Déclencher des alertes par seuil

7 **Créer alerts.py**

À partir du résumé quotidien, on compare chaque valeur maximale aux seuils métier définis pour le type de dispositif.

```
ALERT_THRESHOLDS = {
    "Bottle Filling Machine": {"temperature": 88.0},
}

def check_alerts(daily_summary_collection):
    alerts = []
    for summary in daily_summary_collection.find():
        device_type = "Bottle Filling Machine"  # à enrichir depuis un
registre de devices
        threshold = ALERT_THRESHOLDS.get(device_type,
{}).get("temperature")
        if threshold and summary["max_temperature"] > threshold:
            alerts.append(
                f"⚠  {summary['_id']} a dépassé le seuil de température
"
                f"({summary['max_temperature']:.1f}°C > {threshold}°C) le
{summary['date']}"
            )
    return alerts

if __name__ == "__main__":
    from pymongo import MongoClient
    db = MongoClient("mongodb://localhost:27017/")["aiot_platform"]
    for alert in check_alerts(db["daily_summary"]):
        print(alert)
```

Ce script pourrait être exécuté chaque nuit (tâche planifiée / cron) pour générer un rapport d'alertes quotidien envoyé par email ou Slack.

Résultat attendu

- MongoDB stocke en continu la télémétrie publiée par le dispositif
- Le service d'ingestion découple totalement le dispositif de la base de données
- Un résumé quotidien (min/max/moyenne) est calculé via le pipeline d'agrégation MongoDB
- Des alertes automatiques signalent les dépassements de seuil

Questions de vérification

- ☐ Pourquoi indexer le champ timestamp plutôt qu'un autre champ ?
- ☐ Que se passerait-il si le service d'ingestion s'arrêtait pendant quelques minutes ?
- ☐ Pourquoi utiliser un pipeline d'agrégation MongoDB plutôt qu'une boucle Python pour calculer les agrégats ?

☐ Comment adapteriez-vous alerts.py pour envoyer un email plutôt qu'un simple print() ?

TP 8 — Exposer le dispositif via une API REST (FastAPI)

⏱ Durée estimée : 1h30 Niveau : Intermédiaire / Avancé

Objectifs pédagogiques

- Construire une API REST avec FastAPI exposant la télémétrie stockée dans MongoDB
- Exposer des routes de lecture (historique, dernier état, alertes)
- Envoyer des commandes au dispositif via MQTT depuis un endpoint REST
- Tester l'API avec la documentation interactive générée automatiquement

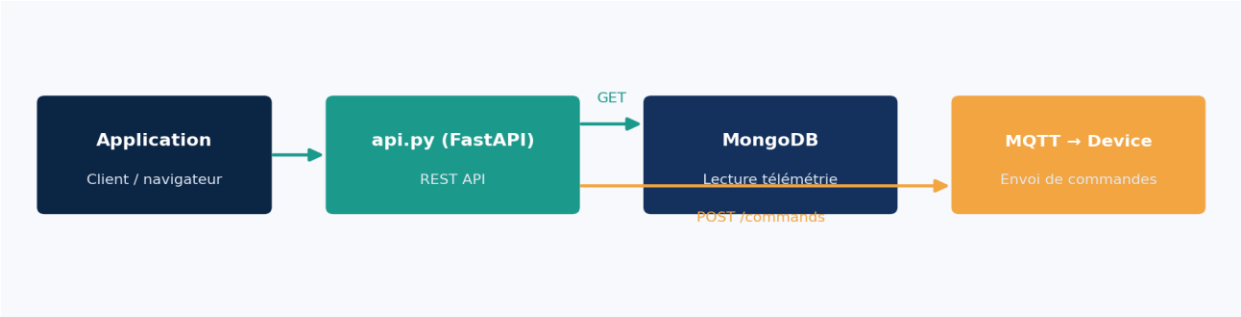
Prérequis

- TP5 terminé (communication bidirectionnelle : commandes + ACK)
- TP7 terminé (ingestion MongoDB opérationnelle)
- pip install fastapi uvicorn pymongo paho-mqtt

Pourquoi une API REST ?

- Jusqu'ici, seules des applications qui "parlent MQTT" ou qui interrogent directement MongoDB peuvent exploiter nos données.
- Une API REST est le langage universel du web : elle permet à n'importe quelle application (site web, app mobile, autre service) d'accéder à la télémétrie et de piloter le dispositif, sans connaître MQTT ni MongoDB.

Schéma de l'architecture cible



L'API REST devient le point d'entrée unique : elle lit la télémétrie dans MongoDB (déjà alimentée par le TP7) et transmet les commandes au dispositif via MQTT (comme au TP5).

Étape 1 — Préparer la couche de données

1 **Créer config.py**

```
DEVICE_CONFIG = {"device_id":
"MCH-007"}

MQTT_CONFIG = {
    "broker_host": "localhost",
    "broker_port": 1883,
    "keepalive": 60,
```

```
    "command_topic_template":
"factory/production-line-
2/machine/{device_id}/command",
    "qos": 0,
}

MONGODB_CONFIG = {
    "uri":
"mongodb://localhost:27017",
    "database":
"aiot_platform",
    "collection": "telemetry",
}

API_CONFIG = {"title":
"Enterprise AIoT Device API",
"version": "1.0.0"}
```

2 **Créer mongo_repository.py — isoler l'accès aux données**

Une bonne pratique d'architecture : l'API ne manipule jamais MongoDB directement, elle passe par un "repository" dédié à la lecture des données.

```
from pymongo import MongoClient, DESCENDING
from config import MONGODB_CONFIG

class TelemetryRepository:
    def __init__(self):
        self.client = MongoClient(MONGODB_CONFIG["uri"],
serverSelectionTimeoutMS=5000)
        self.collection =
self.client[MONGODB_CONFIG["database"]][MONGODB_CONFIG["collection"]]

    def ping(self):
        self.client.admin.command("ping")

    def latest(self, device_id):
        return self.collection.find_one(
            {"device_id": device_id}, sort=[("timestamp", DESCENDING)])

    def history(self, device_id, limit=20):
        limit = max(1, min(limit, 100))
        return list(self.collection.find({"device_id": device_id})
                    .sort("timestamp", DESCENDING).limit(limit))
```

```
    def warnings(self, device_id, limit=20):
        limit = max(1, min(limit, 100))
        return list(self.collection.find({"device_id": device_id,
"state": "WARNING"})
                    .sort("timestamp", DESCENDING).limit(limit))

    def close(self):
        self.client.close()
```

Étape 2 — Publier des commandes MQTT depuis l'API

3

Créer mqtt_commands.py

```
import json, uuid
import paho.mqtt.client as mqtt
from config import MQTT_CONFIG

class MQTTCommandPublisher:
    def __init__(self):
        self.connected = False
        self.client = mqtt.Client(client_id="aiot-rest-
api")
        self.client.on_connect = self._on_connect
        self.client.connect(MQTT_CONFIG["broker_host"],
MQTT_CONFIG["broker_port"], MQTT_CONFIG["keepalive"])
        self.client.loop_start()

    def _on_connect(self, client, userdata, flags, rc):
        self.connected = (rc == 0)

    def send_command(self, device_id, command,
reason=None):
        if not self.connected:
            raise RuntimeError("Broker MQTT non
connecté.")
        correlation_id = f"CMD-
{uuid.uuid4().hex[:10].upper()}"
        payload = {"command": command.upper(),
"correlation_id": correlation_id}
        if reason:
            payload["reason"] = reason
```

```
        topic =
MQTT_CONFIG["command_topic_template"].format(device_id=dev
ice_id)
        self.client.publish(topic, json.dumps(payload),
qos=MQTT_CONFIG["qos"])
        return {"topic": topic, "correlation_id":
correlation_id, "command": command.upper()}

    def close(self):
        self.client.loop_stop()
        self.client.disconnect()
```

Cette classe réutilise exactement le principe du TP5 : publier un JSON structuré sur le topic de commande du dispositif — mais cette fois déclenché par un appel HTTP plutôt que par une saisie clavier.

Étape 3 — Construire l'API FastAPI

4	Créer api.py — routes de lecture

```
from contextlib import asynccontextmanager
from fastapi import FastAPI, HTTPException, Query
from config import API_CONFIG
from mongo_repository import TelemetryRepository
from mqtt_commands import MQTTCommandPublisher

repository = None
command_publisher = None

@asynccontextmanager
async def lifespan(app):
    global repository, command_publisher
    repository = TelemetryRepository()
    repository.ping()
    command_publisher = MQTTCommandPublisher()
    yield
    command_publisher.close()
    repository.close()

app = FastAPI(title=API_CONFIG["title"],
version=API_CONFIG["version"], lifespan=lifespan)

def serialize(doc):
    if not doc:
        return None
```

```
    doc = dict(doc)
    doc["_id"] = str(doc["_id"])
    return doc

@app.get("/health")
def health():
    try:
        repository.ping(); mongo = "healthy"
    except Exception:
        mongo = "unavailable"
    mqtt = "healthy" if command_publisher and
command_publisher.connected else "unavailable"
    return {"status": "healthy" if mongo == mqtt
== "healthy" else "degraded",
            "services": {"mongodb": mongo,
"mqtt": mqtt}}

@app.get("/devices/{device_id}/telemetry/latest")
def latest_telemetry(device_id: str):
    doc = repository.latest(device_id)
    if not doc:
        raise HTTPException(404, f"Aucune
télémétrie pour {device_id}")
    return serialize(doc)

@app.get("/devices/{device_id}/telemetry")
def telemetry_history(device_id: str, limit: int
= Query(20, ge=1, le=100)):
    docs = repository.history(device_id, limit)
    return {"device_id": device_id, "count":
len(docs), "records": [serialize(d) for d in
docs]}

@app.get("/devices/{device_id}/events/warnings")
def warning_events(device_id: str, limit: int =
Query(20, ge=1, le=100)):
    docs = repository.warnings(device_id, limit)
    return {"device_id": device_id, "count":
len(docs), "records": [serialize(d) for d in
docs]}
```

5	**Ajouter la route d'envoi de commande (POST)**

```
from pydantic import BaseModel, Field
```

```
from typing import Optional

class CommandRequest(BaseModel):
    command: str = Field(..., description="START,
STOP ou STATUS")
    reason: Optional[str] = None

@app.post("/devices/{device_id}/commands")
def send_command(device_id: str, request:
CommandRequest):
    command = request.command.upper().strip()
    if command not in {"START", "STOP",
"STATUS"}:
        raise HTTPException(400, "Commande non
supportée. Utilisez START, STOP ou STATUS.")
    try:
        result =
command_publisher.send_command(device_id,
command, request.reason)
    except Exception as exc:
        raise HTTPException(503, str(exc))
    return {"status": "accepted", "message":
"Commande publiée sur MQTT.", **result}
```

GET vs POST : une règle simple

- GET : consulter des données, sans effet de bord (historique, dernier état, alertes).
- POST : déclencher une action qui change quelque chose côté dispositif (ici, une commande MQTT).

Étape 4 — Lancer et tester l'API

6 **Démarrer les services (3 terminaux)**

1. Terminal 1 : lancez le **service d'ingestion** du TP7 (python mqtt_to_mongodb.py)
2. Terminal 2 : lancez le **dispositif à double sens** du TP5 (python device_two_way.py)
3. Terminal 3 : **démarrez l'API** (**uvicorn api:app --reload**)

7 **Ouvrir la documentation interactive**

Rendez-vous sur http://127.0.0.1:8000/docs — FastAPI génère automatiquement une interface de test pour chaque route, sans écrire une seule ligne de HTML.

8 **Scénario de test recommandé**

- ☐ **GET /health** → vérifier que MongoDB et MQTT sont tous les deux "healthy"
- ☐ **GET /devices/MCH-007/telemetry/latest** → dernière mesure connue

- ☐ **GET /devices/MCH-007/telemetry?limit=10** → historique récent
- ☐ **GET /devices/MCH-007/events/warnings** → uniquement les événements WARNING
- ☐ **POST /devices/MCH-007/commands avec {"command": "STOP", "reason": "Test maintenance"}**
- ☐ **Envoyer ensuite START et observer le changement d'état dans le Terminal 2**

Résultat attendu

- Une API REST fonctionnelle exposant la télémétrie stockée en MongoDB
- Des routes de lecture pour le dernier état, l'historique et les alertes
- Une route POST permettant de piloter le dispositif à distance via MQTT
- Une documentation interactive générée automatiquement par FastAPI

Vers la production

- Un système en production ajouterait ici authentification, autorisation, TLS, limitation de débit (rate limiting), gestion des secrets et journalisation d'audit — volontairement omis ici pour rester pédagogique.

Questions de vérification

- ☐ Pourquoi séparer mongo_repository.py de api.py plutôt que d'écrire les requêtes MongoDB directement dans les routes ?
- ☐ Pourquoi la commande est-elle envoyée en POST plutôt qu'en GET ?
- ☐ Que renverrait GET /health si le broker MQTT était arrêté mais que MongoDB fonctionnait normalement ?

TP 9 — AWS IoT Core + DynamoDB : du device au cloud

Durée estimée : 2h00 Niveau : Intermédiaire / Avancé

Objectifs pédagogiques

- Créer un Thing (dispositif), un Type et un Groupe dans AWS IoT Core
- Générer des certificats X.509 et une policy pour sécuriser la connexion
- Connecter un simulateur Python (« BedSideMonitor ») à AWS IoT Core
- Créer une règle IoT Core routant automatiquement les données vers DynamoDB

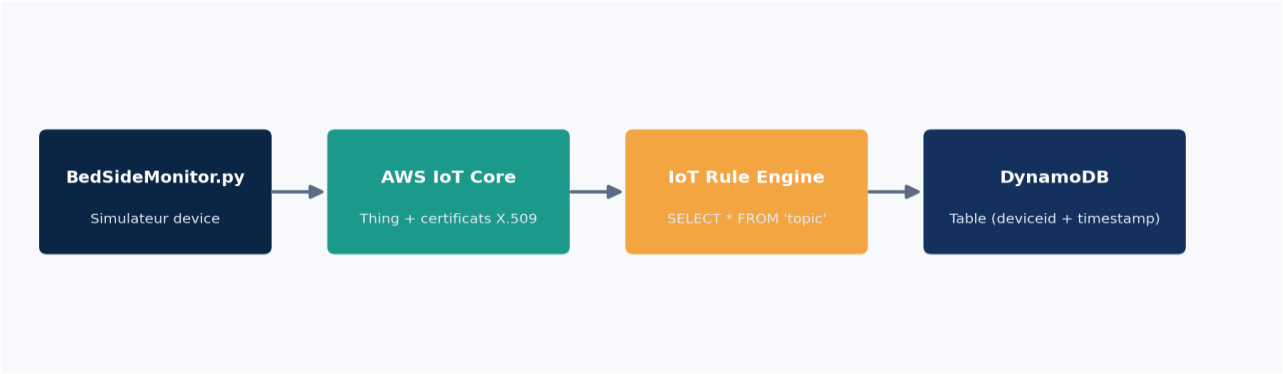
Prérequis

- Un compte AWS (le niveau Free Tier suffit pour ce TP)
- pip install AWSIoTPythonSDK awsiotsdk boto3
- Le script BedSideMonitor.py (simulateur fourni par le formateur)

Important

- Une fois le TP terminé, pensez à supprimer toutes les ressources AWS créées (Thing, certificats, table DynamoDB, rôle IAM) pour éviter des frais imprévus.

Schéma de l'architecture cible



Étape 1 — Créer le Type et le Groupe du dispositif

1 Créer un Thing Type

Dans la console AWS IoT Core → Manage → Types → Create thing type. Nommez-le par exemple BedSideMonitor.

2 Créer un Thing Group

Dans Manage → Thing groups → Create thing group → Create a static thing group. Nommez-le par exemple Hospital (vous pouvez créer des groupes hiérarchiques, ex. Hospital > Department_01).

Étape 2 — Créer la Policy de sécurité

3 Créer une policy IoT Core

Secure → Policies → Create policy. Pour ce TP pédagogique :

Champ	Valeur
Policy effect	Allow
Policy action	All AWS IoT actions (*)
Policy resource	*

En production

– Une policy ne devrait jamais donner un accès total (*) — elle doit restreindre chaque Thing à ses propres topics, selon le principe du moindre privilège (voir cours, Module 3).

Étape 3 — Créer le Thing et ses certificats

4 **Créer un Thing unique**

Manage → Things → Create things → Create a single thing. Renseignez le nom (ex. BSM_A101), associez le Type et le Group créés précédemment.

5 **Générer et télécharger les certificats**

Choisissez l'option recommandée de création automatique des certificats, puis attachez la policy créée à l'étape 2. Téléchargez impérativement les 4 fichiers suivants :

- ☐ Device certificate (xxxx-certificate.pem.crt)
- ☐ Public key file
- ☐ Private key file (xxxx-private.pem.key)
- ☐ Root CA certificate (AmazonRootCA1.pem)

Étape 4 — Connecter le simulateur Python à AWS IoT Core

6 **Adapter BedSideMonitor.py**

Modifiez la ligne d'identification du device dans le script avec le nom de Thing créé :

```
message['deviceid']
= 'BSM_A101'
```

7 **Récupérer l'endpoint AWS IoT Core**

Dans la console AWS IoT Core → Settings, copiez l'endpoint du compte (ex. a2qdlk1319yncw-ats.iot.eu-north-1.amazonaws.com).

8 **Exécuter le simulateur**

```
$ python
BedSideMonitor.py -e
<endpoint> -r
AmazonRootCA1.pem -c
<device-
certificate>.pem.crt
```

```
-k <private-
key>.pem.key -id
client001 -t iot/bsm
```

Le script doit commencer à afficher des messages de télémétrie envoyés vers AWS IoT Core.

Étape 5 — Vérifier la réception avec le client de test MQTT

9 **Utiliser le MQTT Test Client d'AWS IoT Core**

1. Dans la console, ouvrir Test → MQTT test client
2. Saisir le topic utilisé (ex. iot/bsm) et cliquer sur Subscribe
3. Observer les messages JSON arriver en temps réel

Étape 6 — Créer la table DynamoDB

10 **Créer la table**

Service DynamoDB → Create table. Renseignez :

Paramètre	Valeur
Partition key	deviceid
Sort key	timestamp

La combinaison deviceid + timestamp forme la clé primaire de la table.

Étape 7 — Créer le rôle IAM et la règle IoT Core

11 **Créer un rôle IAM**

Service IAM → Roles → Create role → choisir IoT comme cas d'usage → attacher les permissions nécessaires → nommer le rôle (ex. IoTCoreToDynamoDBRole).

12 **Créer la règle de routage**

AWS IoT Core → Message Routing → Rules → Create. Requête SQL de la règle :

```
SELECT * FROM 'iot/bsm'
```

Dans l'action de la règle, choisir DynamoDBv2, sélectionner la table créée et le rôle IAM créé à l'étape précédente.

Étape 8 — Valider le pipeline complet

13 **Relancer le simulateur et observer DynamoDB**

Relancez la commande de l'étape 8. Dans la console DynamoDB, ouvrez la table et cliquez sur Explore table items : les enregistrements doivent apparaître au fil de l'exécution du script.

Pour aller plus loin — Défi : Smart Healthcare

Le même schéma d'architecture (dispositif → AWS IoT Core → DynamoDB) s'applique directement à un cas d'usage santé : un moniteur de chevet (BedSideMonitor) publiant SPO2, température et rythme cardiaque pour plusieurs patients.

Défi (exercice ouvert)

- Simulez 3 moniteurs de chevet (ex. BSM_G101, BSM_G102, BSM_G103) publiant en continu SPO2, température et rythme cardiaque
- Agrégez les mesures par minute dans une table DynamoDB dédiée (clé composite device_id_datatype + timestamp)
- Définissez des règles d'alerte par type de mesure (ex. SPO2 hors de la plage 90-100, rythme cardiaque hors de 75-85) avec un compteur de déclenchement (trigger_count)
- Générez une table d'alertes horodatées, consultable indépendamment des données brutes

Ce défi reprend exactement la structure du TP mais avec plusieurs dispositifs et une logique d'agrégation + détection de règles plus riche — une excellente façon de consolider ce TP avant de passer au traitement en flux (TP10).

Résultat attendu

- Un Thing AWS IoT Core sécurisé par certificats X.509
- Le simulateur Python publie sa télémétrie vers le cloud AWS
- Une règle IoT Core achemine automatiquement chaque message vers DynamoDB
- Les données sont visibles et interrogeables dans DynamoDB

Nettoyage (obligatoire)

- Supprimez la table DynamoDB, le Thing, les certificats, la policy, le rôle IAM et la règle IoT Core une fois le TP terminé.

Questions de vérification

☐ À quoi sert le Device Shadow d'un Thing AWS IoT Core ?

☐ Pourquoi utilise-t-on deviceid + timestamp comme clé primaire dans DynamoDB ?

☐ Quel est le rôle exact du rôle IAM dans la règle de routage ?

TP 10 — Traitement en flux avec AWS Kinesis et détection d'anomalies

Durée estimée : 2h00 Niveau : Avancé

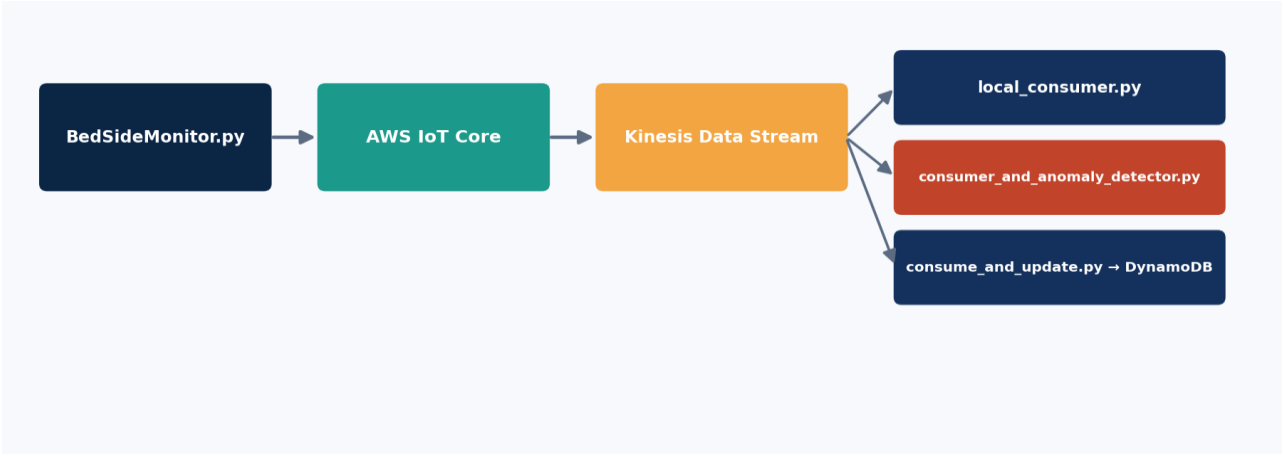
Objectifs pédagogiques

- Créer un flux de données (Data Stream) AWS Kinesis
- Router les données AWS IoT Core vers Kinesis via une règle
- Construire un consommateur Python lisant le flux en temps réel
- Détecter des anomalies et les stocker dans DynamoDB

Prérequis

- TP9 terminé (Thing AWS IoT Core + simulateur opérationnels)
- AWS CLI installé et configuré (aws configure)
- pip install boto3
- Scripts fournis : local_consumer.py, consumer_and_anomaly_detector.py, consume_and_update.py

Schéma de l'architecture cible



Étape 1 — Configurer les accès AWS en ligne de commande

1 **Créer un utilisateur IAM avec clés d'accès**

Service IAM → Users → Create user. Attribuez les permissions nécessaires, puis générez une Access Key et une Secret Access Key pour cet utilisateur.

2 **Configurer AWS CLI**

```
$ aws configure
AWS Access Key ID:
****************
AWS Secret Access Key:
****************
```

```
Default region name: eu-north-1
```

Pourquoi boto3 plutôt qu'AWSIoTPythonSDK ?

- AWSIoTPythonSDK sert à se connecter à IoT Core (dispositif). boto3 est le SDK général AWS utilisé pour interagir avec les autres services : Kinesis, DynamoDB, S3, etc.

Étape 2 — Créer le flux Kinesis

3 **Créer un Data Stream**

Console AWS Kinesis → Create data stream. Donnez-lui un nom (ex. bsm-telemetry-stream) et choisissez le mode de capacité (à la demande, recommandé pour ce TP).

Étape 3 — Router AWS IoT Core vers Kinesis

4 **Créer un rôle IAM pour la règle**

IAM → Create role → cas d'usage IoT → attacher les permissions Kinesis nécessaires.

5 **Créer la règle IoT Core**

AWS IoT Core → Message Routing → Rules → Create. Topic à écouter : iot/bsm (le même que dans le TP9). Dans Rule Action, choisissez Kinesis Data Stream, sélectionnez le flux créé, définissez deviceid comme Partition key, puis sélectionnez le rôle IAM créé.

Étape 4 — Lire le flux avec un consommateur local

6 **Relancer le simulateur BedSideMonitor.py**

Utilisez la même commande que dans le TP9 (avec vos certificats et votre endpoint).

7 **Exécuter le consommateur simple**

Avant exécution, mettez à jour la variable StreamName dans local_consumer.py avec le nom de votre flux Kinesis.

```
$ python local_consumer.py
```

Ce script lit le flux Kinesis et affiche les données reçues en direct sur la console.

Étape 5 — Détecter les anomalies en flux

8 **Exécuter le détecteur d'anomalies**

Mettez à jour StreamName dans consumer_and_anomaly_detector.py, puis lancez :

```
$ python consumer_and_anomaly_detector.py
```

Ce script applique des règles de seuil sur les données du flux (ex. température hors plage normale) et signale les anomalies détectées.

Étape 6 — Stocker les anomalies dans DynamoDB

9 **Créer une table DynamoDB dédiée aux anomalies**

Suivez la même procédure qu'au TP9 (deviceid en clé de partition, timestamp en clé de tri).

10 **Exécuter le script de mise à jour**

Mettez à jour StreamName et le nom de la table dans consume_and_update.py, puis lancez :

```
$ python consume_and_update.py
```

Vérifiez dans la console DynamoDB que les anomalies détectées sont bien enregistrées dans la table.

Étape 7 — Vue d'ensemble du pipeline temps réel

Vous avez maintenant construit un pipeline complet : simulateur → AWS IoT Core → Kinesis Data Stream → consommateurs Python (affichage, détection, stockage) → DynamoDB. En production, ces consommateurs tourneraient sur EC2 ou en fonctions Lambda plutôt qu'en local.

Résultat attendu

- Un flux Kinesis reçoit la télémétrie en provenance d'AWS IoT Core
- Un consommateur Python affiche les données du flux en temps réel
- Un second consommateur détecte automatiquement les anomalies
- Les anomalies détectées sont persistées dans une table DynamoDB dédiée

Nettoyage (obligatoire)

- Supprimez le flux Kinesis, les tables DynamoDB, les règles IoT Core et les rôles IAM créés pour ce TP.

Questions de vérification

- ☐ Pourquoi utiliser deviceid comme clé de partition (Partition key) dans Kinesis ?
- ☐ Quelle est la différence entre Kinesis Data Streams et Kinesis Firehose ?
- ☐ Dans un contexte de production, où feriez-vous tourner les scripts consommateurs plutôt qu'en local ?

TP 11 — TinyML : reconnaissance de gestes sur données de capteur

⏱ Durée estimée : 1h30 Niveau : Intermédiaire / Avancé

Objectifs pédagogiques

- Simuler des données d'accéléromètre représentant différents gestes
- Entraîner un modèle de Machine Learning léger (Decision Tree)
- Appliquer les contraintes propres au TinyML (quantization, taille du modèle)
- Réaliser une inférence en temps réel sur de nouvelles données simulées

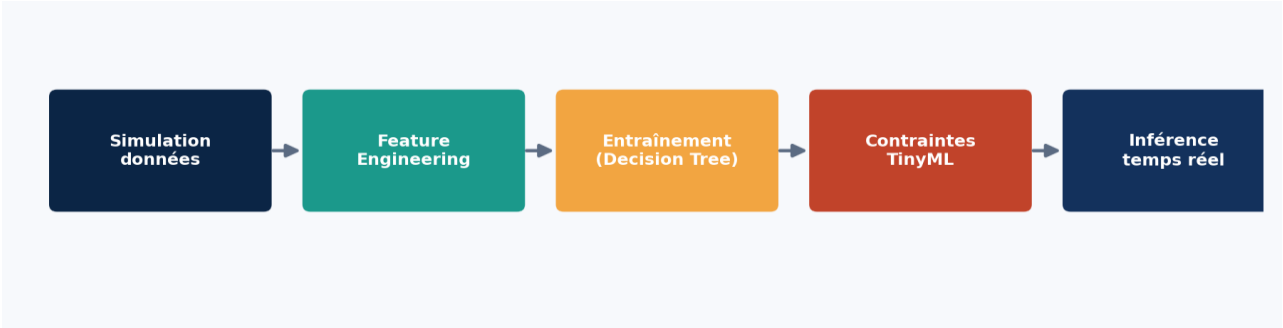
Prérequis

- Python avec scikit-learn, numpy, pandas, matplotlib installés
- pip install scikit-learn numpy pandas matplotlib
- Notions de base du Module 7 (apprentissage supervisé) recommandées

Contexte : construire un système de reconnaissance de gestes

Objectif final : à partir de données de mouvement (accéléromètre X, Y, Z), un modèle léger doit classifier le geste réalisé parmi 4 classes : gauche, droite, haut, bas — puis fonctionner dans les contraintes d'un microcontrôleur.

Schéma du pipeline TinyML



Étape 1 — Simuler les données de capteur

1 Générer un jeu de données synthétique

Chaque geste produit une signature caractéristique sur les axes X, Y, Z. On simule ce comportement avec du bruit gaussien autour de valeurs types par geste.

```
import numpy as np
import pandas as pd

GESTURES = ["left", "right", "up", "down"]
SIGNATURES = {
    "left":  (-1.5, 0.1, 0.2),
    "right": (1.5, 0.1, 0.2),
```

```
    "up":    (0.1, 1.5, 0.2),
    "down":  (0.1, -1.5, 0.2),
}

def generate_samples(n_per_gesture=150):
    rows = []
    for gesture in GESTURES:
        bx, by, bz = SIGNATURES[gesture]
        for _ in range(n_per_gesture):
            x = bx + np.random.normal(0, 0.3)
            y = by + np.random.normal(0, 0.3)
            z = bz + np.random.normal(0, 0.3)
            rows.append([x, y, z, gesture])
    return pd.DataFrame(rows, columns=["x", "y", "z", "gesture"])

data = generate_samples()
print(data.head())
print(data["gesture"].value_counts())
```

Étape 2 — Visualiser et valider les données

2	Tracer X vs Y pour observer la séparation des classes

```
import matplotlib.pyplot as plt

for gesture in GESTURES:
    subset = data[data["gesture"]
== gesture]
    plt.scatter(subset["x"],
subset["y"], label=gesture,
alpha=0.6)

plt.xlabel("Axe X");
plt.ylabel("Axe Y")
plt.legend(); plt.title("Séparation
des gestes (X vs Y)")
plt.savefig("gestures_scatter.png")
plt.show()
```

Vérifiez visuellement que les 4 groupes de gestes forment des nuages de points bien distincts. Une mauvaise séparation à cette étape se traduira par un modèle peu fiable.

Étape 3 — Préparer les données (Train/Test Split + normalisation)

3 **Séparer les jeux d'entraînement et de test**

```
from sklearn.model_selection
import train_test_split
from sklearn.preprocessing
import StandardScaler

X = data[["x", "y",
"z"]].values
y = data["gesture"].values

X_train, X_test, y_train,
y_test = train_test_split(
    X, y, test_size=0.2,
random_state=42, stratify=y)

scaler = StandardScaler()
X_train_scaled =
scaler.fit_transform(X_train)
X_test_scaled =
scaler.transform(X_test)

print(f"Entraînement :
{len(X_train)} échantillons")
print(f"Test : {len(X_test)}
échantillons")
```

Pourquoi normaliser ?

– La normalisation (StandardScaler) évite qu'un axe à plus grande variance ne domine artificiellement le modèle, et améliore la stabilité de l'entraînement.

Étape 4 — Entraîner un modèle léger

4 **Entraîner un Decision Tree (adapté aux contraintes edge)**

Un arbre de décision est volontairement choisi ici : léger, rapide, et facilement convertible en logique embarquée — contrairement à un grand réseau de neurones.

```
from sklearn.tree import DecisionTreeClassifier

model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train_scaled, y_train)
```

```
print("Profondeur réelle de l'arbre :", model.get_depth())
print("Nombre de feuilles :", model.get_n_leaves())
```

Le paramètre max_depth=4 limite volontairement la complexité — un choix essentiel en TinyML où la mémoire disponible se compte en kilo-octets.

Étape 5 — Évaluer le modèle

5	Mesurer accuracy, precision, recall

```
from sklearn.metrics import
accuracy_score,
classification_report

y_pred =
model.predict(X_test_scaled)

print("Accuracy :",
accuracy_score(y_test, y_pred))
print(classification_report(y_test,
y_pred))
```

Un modèle TinyML n'a pas besoin d'être le plus précis possible : il doit être suffisamment fiable tout en restant exploitable sur un dispositif très contraint. Comparez la taille et la complexité du modèle à sa précision obtenue (voir cours, Module 7, sur les métriques d'évaluation).

Étape 6 — Simuler les contraintes TinyML

6	Réduire la précision des paramètres (quantization simplifiée)

Sur un microcontrôleur réel, un modèle est converti en TensorFlow Lite Micro et quantifié en 8 bits. Ici, nous simulons l'effet de cette contrainte en arrondissant les seuils de décision de l'arbre :

```
import numpy as np

# Simule une quantization en arrondissant les seuils internes
thresholds = model.tree_.threshold
quantized_thresholds = np.round(thresholds, 1)
model.tree_.threshold[:] = quantized_thresholds

y_pred_quantized = model.predict(X_test_scaled)
print("Accuracy après quantization simulée :",
      accuracy_score(y_test, y_pred_quantized))
```

Comparez la précision avant/après : une légère baisse est normale et acceptable — c'est le compromis central du TinyML (voir cours, Module 8, techniques d'optimisation).

Étape 7 — Inférence en temps réel

7	Simuler une inférence sur un nouvel échantillon

```
def predict_gesture(x, y, z):
    sample =
scaler.transform([[x, y, z]])
    prediction =
model.predict(sample)[0]
    return prediction

# Simule un nouveau relevé du
capteur en temps réel
live_sample = (1.4, 0.15,
0.25)
result =
predict_gesture(*live_sample)
print(f"Geste détecté :
{result}")
```

Ce bloc représente ce qui, sur un vrai microcontrôleur (ESP32, Arduino), tournerait en continu sur chaque nouvelle lecture de l'accéléromètre.

Mini-défi (optionnel)

Essayez d'ajouter un cinquième geste, ou d'augmenter le bruit gaussien pour observer l'impact sur la précision du modèle. Testez également un modèle de type petit réseau de neurones (MLPClassifier) et comparez sa taille et sa précision à celles du Decision Tree.

Résultat attendu

- Un jeu de données de gestes simulé et visuellement validé
- Un modèle léger entraîné, adapté aux contraintes edge
- Une simulation de l'effet de la quantization sur la précision
- Une fonction d'inférence temps réel opérationnelle

Questions de vérification

- ☐ Pourquoi un Decision Tree est-il plus adapté au TinyML qu'un grand réseau de neurones profond ?
- ☐ Quel est le compromis principal entre précision du modèle et contraintes matérielles ?
- ☐ Quelles étapes supplémentaires seraient nécessaires pour déployer ce modèle sur un vrai microcontrôleur (ESP32) ?

TP 12 — Projet de synthèse — Concevoir une maison intelligente (Smart Home)

Durée estimée : 3h00 (projet) Niveau : Avancé — synthèse

Objectifs pédagogiques

- Concevoir une topologie de topics MQTT complète pour un système multi-dispositifs
- Implémenter des classes Python représentant des dispositifs IoT réels (lampe, climatiseur)
- Construire un serveur Edge coordonnant l'enregistrement et le pilotage des dispositifs
- Mettre en pratique, dans un seul projet, l'ensemble des concepts vus dans les TP précédents

Prérequis

- TP1, TP2 et TP4 terminés (MQTT, simulateur Python, publication/abonnement)
- Broker MQTT Mosquitto opérationnel en local
- Bonne compréhension de la conception de topics (voir TP4, étape 2)

Nature de ce TP

- Contrairement aux TP précédents, celui-ci est un exercice de conception ouvert : il n'y a pas une seule bonne réponse. L'objectif est d'appliquer un raisonnement d'architecte IoT sur un cas concret, en autonomie.

Énoncé du projet

Imaginez une maison avec plusieurs pièces (« rooms »), chacune équipée de dispositifs Smart Light et Smart AC. Un serveur Edge commun interagit de façon bidirectionnelle avec chaque dispositif via MQTT. Le serveur peut également recevoir des commandes d'un consommateur, simulées par un fichier main.py.

Exemple de disposition : BR1 (L1, AC1), BR2 (L2, AC2), LivingRoom (AC3, AC4, L3, L4, L5), Kitchen (L6), Garage (L7).

Composants à réaliser

Composant	Responsabilité
MQTT Server	Broker sans authentification ; IP/port en configuration
LightDevice.py	Lampe multi-intensité (off/low/medium/high) : enregistrement, on/off, changement d'état, réponse à un statut
ACDevice.py	Climatiseur on/off, température 18-32°C ; enregistrement, réponse au statut, gestion des erreurs hors bornes
EdgeServer.py	Stocke les dispositifs enregistrés (en mémoire), accepte les enregistrements, transmet les commandes
main.py	Simulateur consommateur : crée les dispositifs et déclenche les appels vers le serveur

Étape 1 — Concevoir la topologie des topics

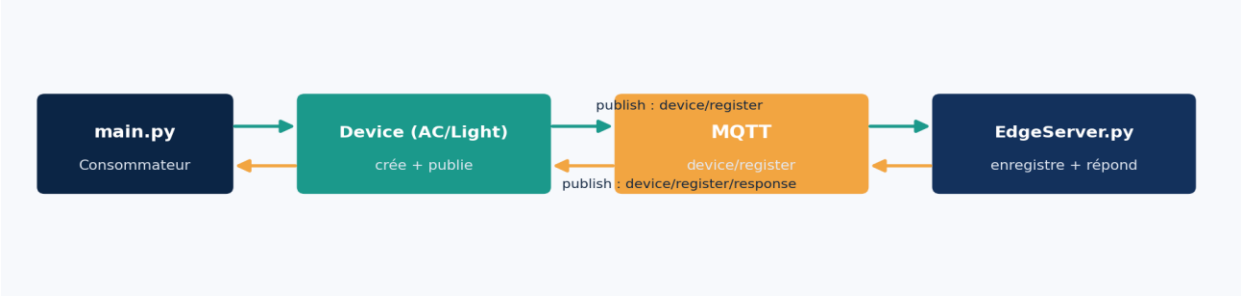
C'est le cœur de l'exercice : un même topic ne doit pas servir à la fois à publier et à recevoir une réponse (cela nuit à la clarté). Réfléchissez à l'équilibre entre deux extrêmes :

- Un topic unique par dispositif → logique simple, mais le serveur doit gérer énormément de topics à grande échelle
- Un seul topic global → très simple côté serveur, mais chaque dispositif reçoit du bruit inutile (tous les messages, filtrés par device_id) et la séparation des accès devient difficile

Une approche hybride est recommandée : chaque dispositif interagit sur ses propres topics tout en écoutant des canaux communs par type et par pièce.

Étape 2 — Comprendre les flux de communication cibles

Flux 1 — Enregistrement d'un dispositif

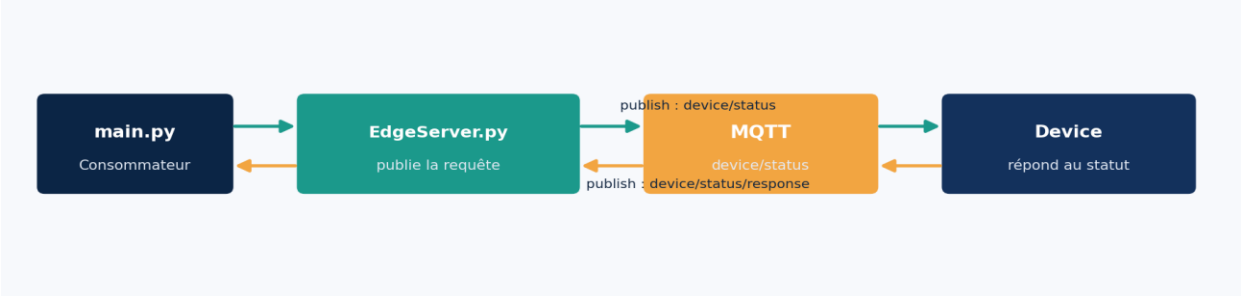


Le dispositif publie sa demande d'enregistrement ; le serveur Edge s'y abonne, l'ajoute à sa liste, puis publie une réponse de confirmation sur un topic dédié, à laquelle le dispositif est abonné.

Flux 2 — Obtenir la liste des dispositifs

Requête directe main.py → EdgeServer.py (pas nécessairement via MQTT) : le serveur retourne la liste de tous les dispositifs enregistrés.

Flux 3 — Lire / modifier le statut (par dispositif, type, pièce ou tous)



Le serveur Edge publie la requête de statut sur le topic concerné ; le ou les dispositifs ciblés répondent sur le topic de réponse associé. Le serveur agrège les réponses avant de les retourner à main.py.

Étape 3 — Tâches à réaliser côté consommateur (main.py)

- ☐ Enregistrer un nouveau dispositif (tous les dispositifs doivent être enregistrés auprès du serveur Edge)
- ☐ Récupérer la liste des dispositifs enregistrés
- ☐ Obtenir le statut d'un dispositif, d'un groupe (par type ou par pièce), ou de tous
- ☐ Modifier le statut d'un dispositif, d'un groupe, ou de tous
- ☐ Modifier une valeur spécifique (intensité de la lampe, température de la climatisation)

Étape 4 — Implémenter l'enregistrement des dispositifs

1 **Logique côté dispositif**

Implémentez l'enregistrement avec au minimum : device_id, device_type, room_type. Le topic MQTT d'enregistrement peut être fixé en dur ou passé au constructeur.

2 **Logique côté serveur**

À la réception d'une demande d'enregistrement, le serveur doit stocker le dispositif dans sa liste interne et répondre avec une confirmation sur un topic dédié.

Étape 5 — Implémenter le statut et les commandes on/off

3 **Méthode get_status exposée à main.py**

Doit accepter en paramètre : un device_id, un device_type (tous les dispositifs de ce type), un room_type (tous les dispositifs de cette pièce), ou 'all'. Le serveur interroge alors tous les dispositifs concernés via MQTT et agrège les réponses avant de les retourner à main.py.

4 **Méthodes set_status (on/off)**

Même logique de ciblage (device_id / type / pièce / tous). Chaque dispositif doit implémenter sa propre méthode on/off retournant succès ou échec.

Étape 6 — Implémenter les commandes de contrôle spécifiques

5 **Contrôle de l'intensité lumineuse**

Changer l'intensité vers une valeur précise (off/low/medium/high), ciblée par device_id, room_type ou 'all' (dispositifs de type lumière uniquement).

6 **Contrôle de la température du climatiseur**

Changer la température vers une valeur comprise entre 18°C et 32°C. Le dispositif doit vérifier les bornes et renvoyer une erreur explicite en cas de dépassement.

Rappel important

- Toute la communication entre EdgeServer et les dispositifs doit transiter par MQTT.
- Une fois qu'une commande est exécutée par un dispositif, le serveur doit renvoyer une confirmation à main.py — cette étape fait partie intégrante de l'exercice.

Livrables attendus

- ☐ ACDevice.py — abonnement aux topics, get/set de la température, gestion des bornes
- ☐ LightDevice.py — abonnement aux topics, get/set de l'intensité lumineuse
- ☐ EdgeServer.py — enregistrement, agrégation get/set, publication des requêtes MQTT
- ☐ main.py — scénario de démonstration couvrant l'ensemble des cas ci-dessus

Résultat attendu

- Une topologie de topics MQTT cohérente et justifiée
- Au moins 3 lampes et 2 climatiseurs, répartis sur au moins 3 pièces, pleinement fonctionnels
- Un scénario de démonstration dans main.py couvrant l'enregistrement, la lecture et la modification de statut

– Un serveur Edge agrégeant correctement les réponses de plusieurs dispositifs

Questions de synthèse

☐ Quelle topologie de topics avez-vous choisie, et pourquoi représente-t-elle le meilleur compromis pour ce cas d'usage ?

☐ Comment votre conception évoluerait-elle avec 10 000 dispositifs répartis sur 500 maisons ?

☐ Quels éléments du cours (QoS, sécurité, persistance MongoDB, Edge AI) intégreriez-vous pour rendre ce système prêt pour la production ?